Bài thực hành 01

Mô hình hóa An toàn Phần mềm bằng Cấu trúc Kripke

Hệ thống SecureSys – Module Access Control

Học phần CSE703093 – An toàn phần mềm
Phenikaa University

Ngày 8 tháng 7 năm 2026

Tóm tắt nội dung

Bài thực hành này giới thiệu khái niệm **cấu trúc Kripke** (Kripke structure) – nền tảng hình thức của mô hình hóa và kiểm chứng hệ thống (model checking). Sinh viên sẽ mô hình hóa module Access Control (kiểm soát truy cập cửa ra vào) của hệ thống SecureSys, kiểm chứng các thuộc tính an toàn bằng thuật toán duyệt không gian trạng thái (BFS), và đối chiếu với một cài đặt thực thi (C) để phát hiện sự khác biệt giữa **mô hình** và **cài đặt**.

Mục lục

1 Kiến thức nền tảng **2**
1.1 Cấu trúc Kripke là gì? 2
1.2 Bối cảnh: Module Access Control 2

2 Dữ liệu (Dataset) **2**

3 Cài đặt **3**
3.1 Python – Kiểm chứng mô hình (Model Checking) 3
3.2 C – Mô phỏng thực thi (Runtime Simulator) 3

4 Kết quả thực nghiệm **3**
4.1 Kết quả kiểm chứng mô hình 3
4.2 Kết quả mô phỏng runtime (C) 4

5 Kết quả kiểm thử tự động **4**

6 Câu hỏi thảo luận **5**

7 Tài liệu tham khảo **5**

1 Kiến thức nền tảng

1.1 Cấu trúc Kripke là gì?

Một **cấu trúc Kripke** là một bộ bốn thành phần:

$$M = (S, S_0, R, L)$$

trong đó:

- S – tập hữu hạn các **trạng thái** (states);
- $S_0 \subseteq S$ – tập **trạng thái khởi tạo** (initial states);
- $R \subseteq S \times S$ – **quan hệ chuyển trạng thái** (transition relation);
- $L : S \to 2^{AP}$ – **hàm gán nhãn** (labeling function), với AP là tập **mệnh đề nguyên thủy** (atomic propositions).

Các thuộc tính an toàn được biểu diễn bằng logic thời gian (temporal logic), đơn giản hóa trong bài này với 3 dạng:

- $AG\ \neg p$ – **an toàn** (safety): ”không bao giờ” p xảy ra trên bất kỳ đường thi hành nào.
- $AG\ (p \to q)$ – **an toàn có điều kiện**: mỗi khi p đúng thì q phải đúng.
- $EF\ p$ – **khả thi** (reachability): tồn tại ít nhất một đường đi dẫn đến trạng thái thỏa p.

1.2 Bối cảnh: Module Access Control

Hệ thống khóa cửa điện tử có các sự kiện: `insert_card`, `auth_success`, `auth_fail`, `auth_fail_3rd_time`, `door_closed_timeout`, `tamper_sensor`, `operator_reset`. Ba thuộc tính an toàn cần kiểm chứng:

Mã	Mô tả	Loại
P1	Không bao giờ `unlocked` mà chưa `authenticated`	$AG\neg p$
P2	Mỗi khi `tamper`, phải `alarm`	$AG(p \to q)$
P3	Tồn tại đường đi đến `unlocked`	$EF\ p$

2 Dữ liệu (Dataset)

Hai bộ dữ liệu JSON mô tả cấu trúc Kripke được cung cấp:

- `dataset/access_system_model.json` – phiên bản **SAFE** (đúng đặc tả).

- `dataset/access_system_model_BUGGY.json` – phiên bản **BUGGY**, chứa một cạnh chuyển tiếp sai (`timeout_glitch`) mô phỏng lỗi logic thực tế (ví dụ: điều kiện đua – race condition – bỏ qua bước xác thực).

3 Cài đặt

3.1 Python – Kiểm chứng mô hình (Model Checking)

Lớp `KripkeStructure` đọc dữ liệu JSON, xây dựng đồ thị trạng thái, và cung cấp các hàm kiểm chứng thuộc tính dựa trên **BFS**:

```python
def check_AG_not(self, formula: str):
    """AG !formula : khong trang thai nao dat duoc thoa 'formula'."""
    reachable = self.reachable_states()
    violating = [s for s in reachable if self._eval_formula(s, formula)]
    if not violating:
        return True, None
    path = self._find_path(lambda s: self._eval_formula(s, formula))
    return False, path
```

Listing 1: Hàm kiểm chứng $AG\neg p$ (trích từ kripke_model.py)

3.2 C – Mô phỏng thực thi (Runtime Simulator)

Chương trình `access_simulator.c` đọc cùng dữ liệu (dùng CSV) và mô phỏng thi hành theo một chuỗi sự kiện cụ thể, áp dụng nguyên tắc **fail-closed**: sự kiện không hợp lệ bị từ chối thay vì gây hành vi không xác định.

```c
if (snxt && has_label(snxt, "unlocked") && !has_label(snxt, "authenticated"
    )) {
    printf("  [CANH BAO AN TOAN] Trang thai '%s' la 'unlocked' nhung "
           "KHONG 'authenticated' - vi pham thuoc tinh P1!\n", snxt->id);
}
```

Listing 2: Kiểm tra vi phạm runtime trích từ access_simulator.c

4 Kết quả thực nghiệm

4.1 Kết quả kiểm chứng mô hình

Thuộc tính	Mô hình SAFE	Mô hình BUGGY	Ghi c
P1 (không unlock chưa auth)	PASS	**FAIL**	Phát hiện counterex
P2 (tamper $\rightarrow$ alarm)	PASS	PASS (vô tình)	Trạng thái xử lý tamper
P3 (khả thi unlock)	PASS	PASS	Không

Nhận xét sư phạm quan trọng: thuộc tính P2 ”PASS” trên bản BUGGY không có nghĩa là hệ thống an toàn – đó là một dạng **PASS vô tình (vacuous pass)** vì trạng thái xử lý `tamper` đã trở thành **không thể đạt được (unreachable)** do lỗi – bản thân điều này cũng là một lỗi (dạng ”dead code an toàn” nhưng che giấu một đường xử lý quan trọng).

4.2 Kết quả mô phỏng runtime (C)

Khi chạy `access_simulator` với chuỗi sự kiện `insert_card timeout_glitch` trên bộ dữ liệu BUGGY, chương trình in ra **cảnh báo an toàn runtime** khớp chính xác với counterexample mà Python đã tìm được bằng kiểm chứng tĩnh – chứng minh tính nhất quán giữa hai tầng kiểm tra.

5 Kết quả kiểm thử tự động

File `tests/test_kripke.py` thực hiện 5 test:

1. `test_safe_model_all_properties_hold` – PASS
2. `test_buggy_model_violates_p1` – PASS
3. `test_buggy_model_reachability_still_holds` – PASS
4. `test_c_simulator_detects_runtime_violation` – PASS
5. `test_c_simulator_safe_sequence_no_warning` – PASS

Kết quả: 5/5 test PASS.

Mục tiêu học tạp

- Biểu diễn được một hệ thống thực dưới dạng cấu trúc Kripke.
- Cài đặt thuật toán duyệt không gian trạng thái bằng BFS.
- Kiểm chứng các dạng thuộc tính $AG\neg p$, $AG(p \to q)$, $EF\ p$.
- Phân biệt lỗi trong mô hình (tĩnh) với lỗi trong cài đặt (động).

Yêu cạu hoàn thiện

1. Mô tả bằng lời cấu trúc Kripke của module Access Control (S, S_0, R, L).
2. Log chạy kiểm chứng trên cả hai dataset.
3. Một thuộc tính CTL mới tự viết và kết quả.
4. Log `make test` toàn bộ PASS.

Yêu cạu kẹt quạ

Tất cả 5 test trong `tests/test_kripke.py` PASS; phát hiện đúng đường đi phản chứng $S_0 \to S_1 \to S_5$ cho vi phạm P1.

6 Câu hỏi thảo luận

1. Tại sao P2 ”PASS” trên bản BUGGY lại không đáng tin cậy?
2. Nếu thêm trạng thái mới với nhãn `unlocked` và `alarm` đồng thời, điều gì xảy ra với thuộc tính hiện có?
3. So sánh chi phí tính toán của BFS ($O(|S| + |R|)$) với kiểm chứng bằng liệt kê vét cạn khi $|S|$ rất lớn.

7 Tài liệu tham khảo

- Đề cương CSE703093 – An toàn phần mềm, Chương 1.
- E. Clarke, O. Grumberg, D. Peled, *Model Checking*, MIT Press.